

Make 도구

- make 유틸리티 | 1.
- 간단한 Makefile | 2.
- 매크로(Macro) 와 확장자(Suffix) 규칙 | 3.
- Makefile를 작성할 때 알면 좋은 것들 | 4.
- make 중요 옵션 정리 | 5.
- Makefile의 실제 예제 | 6.
- make의 에러들 | 7.

1. make 유틸리티

1.1 make 도구

영어 사전에서 make란 뜻은 누구나 알듯이 '만들다'라는 뜻의 동사이다. 그럼 make유틸리티는 왜 이름이 make인지 알 필요가 있을 것 같다. man으로 찾아보면 make에 대해 다음과 같이 설명하고 있다.

make - maintain, update, and regenerate groups of programs

우리말로 하면 make는 프로그램 그룹을 유지하는데 필요한 유틸리티이다. make유틸리티의 목적은 프로그램 그룹 중에서 어느 부분이 새롭게 컴파일되어야 하는지를 자동적으로 판단해서 필요한 커맨드(gcc 등)를 이용해서 그들을 재컴파일 시킨다고 되어 있다.

make는 일련의 프로그램 개발에만 쓰이지 않고, 컴파일러처럼 일종의 명령어 방식으로 처리되는 모든 곳에서 쓰일 수가 있다. 가령 LaTeX와 같은 경우도 .tex 파일에서 .dvi 파일을 만들고 다시 .ps 파일로 만드는 과정을 make를 사용해서 간단하게 만들 수가 있다.

쉽게 말하면 다음과 같은 경우에 make를 쓰면 유리하다.

1. 입력 파일이 바뀌면 자동적으로 결과 파일이 바뀌기를 원할 때, 기왕이면 좀 지능적으로 일이 수행되기를 바라는 경우
2. 위의 LaTeX 파일처럼 자동적으로 프로그램이 수행이 되기를 바라는 경우(배치(batch)의 개념)

=> make는 위의 두 가지 개념을 모두 포함하고 있다고 본다.

make는 보통 Makefile, makefile 중에서 하나가 있으면 그 파일을 읽게 된다. 하지만 일반적으로 Makefile을 추천하게 되는데, 그 이유는 우선 makefile은 보통 소스 파일에 묻혀서 잘 안보이게 되기 때문이다.

Makefile은 make가 이해할 수 있도록 일종의 쉘 스크립트 언어같이 되어 있다(makefile database라 하기도 한다). 이 파일에는 결과 파일을 생성시키기 위한 파일들간의 관계, 명령어 등을 기술있다.

1.2 make의 필요성

본 과정에서는 make의 사용을 프로그램 개발과 유지 쪽으로 국한시키기로 한다. 보통 라인 수가 많아지면 여러 개의 파일로 나누어 (모듈로 나누어) 개발을 하게 된다. 이들은 알게 모르게 서로 관계를 가지고 있는데, 어느 하나를 필요에 의해 바꾸게 되었을 때 그 파일에 있는 함수를 이용하는 다른 파일도 새롭게 컴파일되어야 한다.

하지만 파일 수가 많은 경우 이를 일일이 컴파일을 하게 될 때, 그 불편함과 함께 컴파일하지 않아도 될 것도 컴파일을 하게 될 수도 있고, 컴파일해야 할 것도 미처 못하게 되는 경우가 있다(링크 에러의 원인이 되기도 하는데 에러의 원인을 제대로 찾기가 힘이 든다).

앞에서도 얘기했듯이 이런 상황에서 지능적으로 관계 있는 것만 새롭게 갱신을 할 필요가 있을 때 make파일은 빛을 발하게 된다.

2. 간단한 Makefile

2.1 Makefile 내부 구조

Makefile은 기본적으로 아래와 같이 목표(target), 의존 관계(dependency), 명령(command)의 세개로 이루어진 기본적인 규칙(rule)들이 계속적으로 나열되어 있다고 봐도 무방하다. make가 지능적으로 파일을 갱신하는 것도 모두 이 간단한 규칙에 의하기 때문이다.

```
target : dependency ...  
    command  
    ...  
    ...
```

여기서 목표(target) 부분은 명령(command)이 수행이 되어서 나온 결과 파일을 지정한다. 당연히 목적 파일(object file)이나 실행 파일이 될 것이다.

명령(command)부분에 정의된 명령들은 의존 관계(dependency)부분에 정의된 파일의 내용이 바뀌었거나, 목표 부분에 해당하는 파일이 없을 때 이곳에 정의된 것들이 차례대로 실행이 된다. 일반적으로 셸에서 쓸 수 있는 모든 명령어들을 사용할 수가 있으며 bash에 기반한 쉘 스크립트도 지원한다.

=> 참고: 참고로 목표 부분에는 결과 파일만 올 수 있는 것이 아니고, 보통 make clean 에서와 같이 간단한 레이블(label) 기능을 제공하기도 한다.

=> 명령 부분은 꼭 **TAB** 글자로 시작해야 한다. 그냥 빈칸 등을 사용하면 make 실행 중에 에러가 난다. (중요). make가 명령어인지 아닌지를 TAB 가지고 구별하기 때문이다.

2.2 makefile 예제

간단한 Makefile을 만들어 본다. 우리가 만들려고 하는 프로그램은 main.c read.c write.c로 구성되어 있고 모두 io.h라는 헤더 파일을 사용한다고 가정한다. 이들을 각각 컴파일해서 test라는 실행 파일을 생성시킨다.

```
$ gcc -c main.c
$ gcc -c read.c
$ gcc -c write.c

$ gcc -o test main.o read.o write.o
```

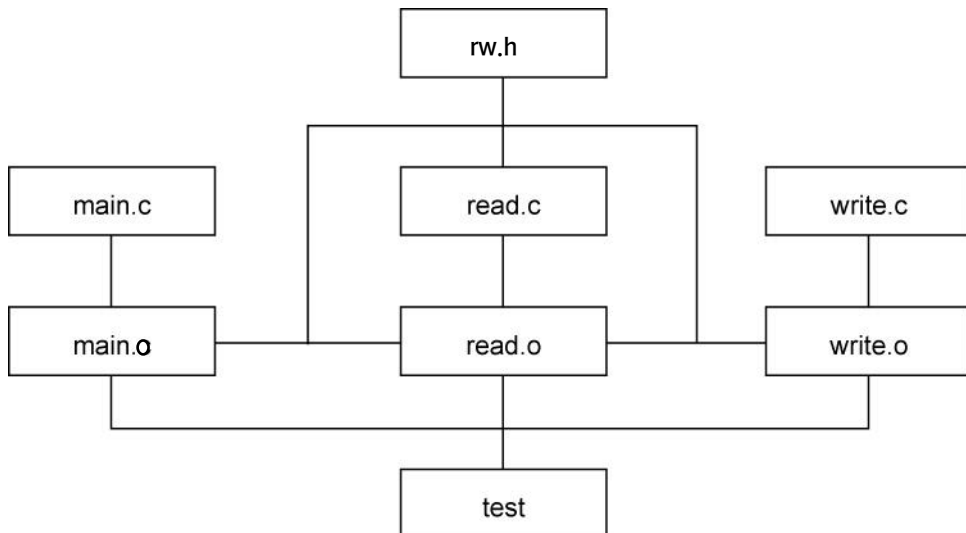
그리고, 아래는 위와 똑같은 일을 수행하는 Makefile의 내용이다.

```
$ cat Makefile

test : main.o read.o write.o
    gcc -o test main.o read.o write.o

main.o : rw.h main.c
    gcc -c main.c
read.o : rw.h read.c
    gcc -c read.c
write.o : rw.h write.c
    gcc -c write.c
```

make는 Makefile의 내용을 보고, 내부적으로 어떻게 파일들이 의존하고 있는지 조사한다. 위의 Makefile을 바탕으로 의존 관계를 그림으로 나타내 보면 아래와 같다.



위의 그림에서 보면 test 가 만들어지기 위해서는 main.o read.o write.o가 필요하게 각각의 목적 파일들은 모두 자신의 소스 파일과 rw.h 에 의존함을 알 수가 있다.

가령 main.c를 고쳤다고 생각한다면 main.o가 컴파일되어 다시 생기고, test 도 다시 링크되어 갱신된다. 만약 rw.h가 바뀌었다고 가정하면 모든 파일들이 컴파일되어서 목적 파일이 생기고, 그것들이 링크가 되어 test가 생긴다.

위와 같이 파일들을 구성한 다음 Makefile을 실행시켜 보자. Makefile의 실행은 그냥 make라고만 치면 된다.

[예제 1]

```

$ make
gcc -c main.c
gcc -c read.c
gcc -c write.c
gcc -o test main.o read.o write.o <- OK
  
```

2.3 매크로의 사용

간단한 매크로 기능을 사용해 보자. main.o read.o write.o라는 것을 OBJECTS 라는 매크로로 바꾸어보자.

[예제 2]

```
OBJECTS = main.o read.o write.o
```

```
test : $(OBJECTS)
```

```
    gcc -o test $(OBJECTS)
```

```
main.o : rw.h main.c
```

```
    gcc -c main.c
```

```
read.o : rw.h read.c
```

```
    gcc -c read.c
```

```
write.o: rw.h write.c
```

```
    gcc -c write.c
```

위에서 보드시피 매크로는 그냥 프로그램 짤 때와 같이 사용해서 값을 대입한다. 대신 사용할 때는 반드시 \$(..) 안에 넣어서 사용한다. \$(..)은 매크로 치환을 위한 특수한 방법이다. 매크로의 사용법은 위와 같이 간단하므로 다양하게 정의해서 사용할 수 있다. 매크로에 대한 자세한 설명은 다음 절에서 언급하기로 한다.

2.4 레이블의 사용

목표(Target) 부분에 해당하는 부분이 그냥 레이블과 같이 사용될 수도 있다고 이미 설명하였다. 예제 2 에 목적 파일들을 모두 삭제하는 명령어를 추가하기로 한다.

[예제 3]

```
OBJECTS = main.o read.o write.o
```

```
test : $(OBJECTS)
```

```
    gcc -o test $(OBJECTS)
```

```
main.o : rw.h main.c
```

```
    gcc -c main.c
```

```
read.o : rw.h read.c
```

```
    gcc -c read.c
```

```
write.o: rw.h write.c
```

```
    gcc -c write.c
```

```
clean :
```

```
    rm $(OBJECTS)
```

레이블로 사용될 때는 당연히 의존 관계 부분은 없어도 된다. 그리고 clean을 실행시키려면 아래와 같이 한다.

```
$ make clean
```

```
rm main.o read.o write.o <- OK
```


3. 매크로와 확장자 규칙

3.1 매크로란?

앞에서 매크로에 대해서 간단하게 살펴보았다. 프로그램을 작성해본 사람이나 로터스, 한글, 엑셀 등의 모든 패키지에서 매크로라는 것을 사용하게 된다. 은연중에 매크로의 정의는 대충 짐작하고 있을 것이다. 이미 알고 있는바와 같이 매크로는 특정한 코드를 간단하게 표현한 것에 지나지 않는다. Makefile에서 사용되는 매크로는 비교적 그 사용법이 간단하기 때문에 금방 익혀서 사용할 정도가 된다.

매크로의 정의는 프로그램을 작성할 때 변수를 지정하는 것처럼 하면 된다. 그리고, 매크로를 사용하기 위해서는 \$(..)을 이용하면 된다. 아래는 매크로의 간단한 예제이다.

⇒ 참고: 매크로의 사용에서 \${..}, \$(..), \$..를 모두 사용할 수 있습니다. 그러나 대부분은 \$(..)을 사용하라고 권함.

[예제 4]

```
OBJ = main.o read.o write.o

test : $(OBJ) <- (1)
    gcc -o test $(OBJ)

.....
```

매크로는 사실상 복잡한 것을 간단하게 표시한 것에 지나지 않는다. (1) 번을 매크로를 안 쓰고 표현한다면 아마 아래와 같이 될 것이다.

[예제 5]

```
test : main.o read.o write.o
    gcc -o test main.o read.o write.o
```

.....

Macro makes Makefile happy.

3.2 미리 정해져 있는 매크로

여러분들보다 머리가 약간 더 좋은 사람들이 make 라는 것을 만들면서 미리 정해 놓은 매크로들이 있다. 'make -p' 라고 입력해 보면 make에서 미리 세팅되어 있던 모든 값들(매크로, 환경 변수(environment) 등등)이 엄청 스크롤 된다. 이 값들을 보고 미리 주눅 들 필요는 없다. 어차피 대부분의 내용들은 우리가 재정의 해주어야 하기 때문에 결론적으로 말하면 우리가 모두 작성한다고 생각하는 것이 마음이 편하다.

아래에는 대부분이 Linux/Unix 계열의 make에서 미리 정해져 있는 매크로들 중에 몇 가지만 나열해 본 것이다.

[예제 6] Predefined Macro

```
test : main.o read.o write.o
      gcc -o test main.o read.o write.o
```

.....

ASFLAGS = <- as 명령어의 옵션 세팅

AS = as

CFLAGS = <- cc 의 옵션 세팅

CC = gcc (= cc)

CPPFLAGS = <- g++ 의 옵션

CXX = g++

LDLFLAGS = <- ld 의 옵션 세팅

LD = ld

LFLAGS = <- lex 의 옵션 세팅

LEX = lex

```
YFLAGS = <- yacc 의 옵션 세팅  
YACC = yacc  
MAKE_COMMAND = make
```

=> 참고: 직접 `make -p`를 해서 한번 확인해 본다. 과연 `make`는 내부적으로 어떤 변수들을 사용하고 있는지 알아본다. 매크로는 관습적으로 대문자로 작성되니까 이점에 유의해서 확인해 본다. `make`는 쉘상에서 정의한 환경 변수값들을 그대로 이용한다

위에 열거한 매크로는 `make`에서 정의된 매크로중 그야말로 일부에 지나지 않는다. 하지만 프로그램을 작성함에 있어 가장 많이 사용하게 될 매크로 들이다. 이들 매크로는 사용자에 의해 재정의 가능하다. 가령 `cc`의 옵션 중에 디버그 정보를 표시하는 `'-g'` 옵션을 넣고 싶다면, 아래와 같이 재정의 한다.

```
CFLAGS = -g
```

`make`에서 위에 나온 것들을 왜 미리 정해 두었을까?

`make`에서 이들 매크로를 제공하고 있는 이유는 내부적으로 이들 매크로를 사용하게 되기 때문이다. 어떻게 이용하는지는 확장자 규칙(Suffix rule)을 설명하면서 해답을 제공할 것이다. 이제 예제 4 의 Makefile을 매크로를 이용하여 깔끔하게(?) 작성해 보자.

[예제 7]

```
OBJECTS = main.o read.o write.o  
SRCS = main.c read.c write.c <- 없어도 됨  
  
CC = gcc <- gcc 로 세팅  
CFLAGS = -g -c <- gcc 의 옵션에 -g 추가  
  
TARGET = test <- 결과 파일을 test 라고 지정  
  
$(TARGET) : $(OBJECTS)  
    $(CC) -o $(TARGET) $(OBJECTS)
```

```
clean :  
    rm -rf $(OBJECTS) $(TARGET) core  
  
main.o : rw.h main.c    <- (1)  
read.o : rw.h read.c  
write.o: rw.h write.c
```

위의 Makefile 을 동작시켜 보자.

```
$ make cc -g -c main.c -o main.o  
gcc -g -c read.c -o read.o  
gcc -g -c write.c -o write.o  
gcc -o test main.o read.o write.o    <- OK  
  
$ make clean  
rm -rf main.o read.o write.o test core    <- OK
```

그런데 여기서 .c 파일을 .o 파일로 바꾸는 부분이 없는데 어떻게 컴파일이 되었을까?

앞에서 CFLAGS 같은 매크로는 make 파일의 내부에서 이용된다고 하였다. 그렇다면 make는 과연 어디에서 이용을 할까? 바로 컴파일하는 곳에서 이용을 하는 것이다. 따라서 우리는 CFLAGS를 셋팅해 주기만 하면 make가 알아서 컴파일을 수행하는 것이다.

=> 참고: 확장자 규칙에서 다시 한번 자세히 설명한다

(1) 에 해당하는 부분은 어떤 파일이 어디에 의존하고 있는지를 표시해 주기 위해서 꼭 필요하다. .c 파일을 컴파일하는 부분은 일괄적인 루틴으로 작성할 수 있기 때문에 이들 파일간의 의존 관계(dependency)를 따로 표시해 주어야 한다.

=> 참고: 파일간의 의존 관계를 자동으로 작성해 주는 유틸리티가 있다. 이것은 다음 절에서 다루기로 한다.

3.3. 확장자 규칙

확장자 규칙이란 간단히 말해서 파일의 확장자를 보고, 그에 따라 적절한 연산을 수행시키는 규칙이라고 말할 수 있다. 가령 .c 파일은 일반적으로 C 소스 코드를 가리키며, .o 파일은 목적 파일(Object file)을 말하고 있다. 그리고 당연히 .c 파일은 컴파일되어서 .o 파일이 되어야 하는 것이다.

여기서 한가지 매크로가 등장하게 된다. .SUFFIXES 라고 하는 매크로인데 우리가 make 파일에게 주의 깊게 처리할 파일들의 확장자를 등록해 준다고 이해하면 될 것이다.

```
.SUFFIXES : .c .o
```

위의 표현은 '.c' 와 '.o' 확장자를 가진 파일들을 확장자 규칙에 의거해서 처리될 수 있도록 해준다. .SUFFIXES 매크로를 이용한 예제를 살펴보자.

[예제 8]

```
.SUFFIXES : .c .o

OBJECTS = main.o read.o write.o
SRCS = main.c read.c write.c

CC = gcc
CFLAGS = -g -c

TARGET = test

$(TARGET) : $(OBJECTS)
    $(CC) -o $(TARGET) $(OBJECTS)

clean :
    rm -rf $(OBJECTS) $(TARGET) core

main.o : rw.h main.c
read.o : rw.h read.c
write.o : rw.h write.c
```

위의 Makefile 을 동작시켜 보자.

```
$ make
gcc -g -c main.c -o main.o
gcc -g -c read.c -o read.o
gcc -g -c write.c -o write.o
gcc -o test main.o read.o write.o  <- OK
```

확장자 규칙에 의해서 make는 파일들간의 확장자를 자동으로 인식해서 필요한 작업을 수행한다. 즉 아래의 루틴이 자동적으로 동작하게 된다.

```
.c.o :
    $(CC) $(CFLAGS) -c $(  
-o $@
```

우리가 .SUFFIXES : .c .o 라고 했기 때문에 make 내부에서는 미리 정의된 .c (C 소스 파일)를 컴파일해서 .o (목적 파일)를 만들어 내는 루틴이 자동적으로 동작하게 되어 있다. CC와 CFLAGS 도 우리가 정의한 대로 치환될 것임은 의심할 여지가 없다.

make 내부에서 기본적으로 서비스를 제공해 주는 확장자들의 리스트를 열거해 보면 아래와 같다. 각 확장자에 따른 자세한 설명은 생략한다.

```
.out .a .ln .o .c .cc .C .p .f .F .r .y .l .s .S .mod .sym .def .h .info .dvi .tex .texinfo .texi .txinfo .w .ch .web .sh .elc .el
```

Makefile내부에서 .SUFFIXES 매크로의 값을 세팅해 주면 내부적으로 정의된 확장자의 연산이 동작을 하게 된다. 따라서 확장자 규칙은 make가 어느 확장자를 가진 파일들을 처리할 것인가를 정해 주는 것이라고 생각할 수 있다.

그러나 이것은 make에서 자동적으로 확장자를 알아서 해주는 것이 좋긴 하지만, 일부러 위의 .c.o 에 해당되는 부분을 그냥 정의해서 쓰는게 더 좋다.

예제 8 의 내용을 약간 바꾸어 보자.

[예제 9]

```

.SUFFIXES : .c .o

OBJECTS = main.o read.o write.o
SRCS = main.c read.c write.c

CC = gcc
CFLAGS = -g -c
INC = -I/home/user/include  <- include 패스 추가

TARGET = test

$(TARGET) : $(OBJECTS)
    $(CC) -o $(TARGET) $(OBJECTS)

.c.o :  <- 직접 확장자 규칙을 구현
    $(CC) $(INC) $(CFLAGS) $<

clean :
    rm -rf $(OBJECTS) $(TARGET) core

main.o : rw.h main.c
read.o : rw.h read.c
write.o : rw.h write.c

```

```

$ make
gcc -I/home/user/include -g -c main.c
gcc -I/home/user/include -g -c read.c
gcc -I/home/user/include -g -c write.c
gcc -o test main.o read.o write.o <- OK

```

예제 8 과 예제 9 의 차이는 .c .o 부분을 누가 처리하느냐이다. 그리고 예제 9에서는 INC 라는 매크로를 추가시켜서 컴파일할 때 이용하도록 하였다.

3.4 내부 매크로

make에서는 미리 정의되어 있는 내부 매크로가 있다.

[예제 10]

```
$*  <- 확장자가 없는 현재의 목표 파일(Target)
$@  <- 현재의 목표 파일(Target)
$<  <- 현재의 목표 파일(Target)보다 더 최근에 갱신된 파일 이름
$?  <- 현재의 목표 파일(Target)보다 더 최근에 갱신된 파일이름
```

각 내부 매크로에 대한 예를 보기로 한다.

```
main.o : main.c io.h
gcc -c $*.c
```

\$* 는 확장자가 없는 현재의 목표 파일이므로 \$* 는 결국 main 에 해당한다.

```
test : $(OBJ)S
gcc -o $@ $*.c
```

\$@는 현재의 목표 파일이다. 즉 test에 해당된다.

```
.c.o :
gcc -c $< (또는 gcc -c $*.c)
```

\$< 는 현재의 목표 파일보다 더 최근에 갱신된 파일 이름이라고 하였다. .o 파일보다 더 최근에 갱신된 .c 파일은 자동적으로 컴파일이 된다. 가령 main.o를 만들고 난 다음에 main.c를 갱신 하게 되면 main.c는 \$<의 작용에 의해 새롭게 컴파일이 된다.

=> 참고: Makefile 파일을 작성하고, make 명령을 인수없이 실행하면 make는 Makefile의 내용을 살펴보다가 첫 번째 목표 파일에 해당되는 것을 실행시킨다. 따라서 위의 예제에서는 make test 라고 해도 같은 결과를 얻게 된다. 반면 clean에 해당하는 부분을 윗부분에 두게

되면 make는 항상 make clean을 수행하게 됩니다.

\$ make <- make clean 이 실행됨

```
rm -rf main.o read.o write.o test core
```

\$make test <- 강제로 test 가 생성

```
gcc -I/home/user/include -g -c main.c
```

```
gcc -I/home/user/include -g -c read.c
```

```
gcc -I/home/user/include -g -c write.c
```

```
gcc -o test write.c main.o read.o write.o <- OK
```

4. Makefile 작성할 때 알아두면 좋은 것들

Makefile을 작성할 때 기본적으로 알고 있으면 유익한 것들을 기술한다. 이전 강좌의 내용을 대체로 이해하고 있다면 좋은 팁이 될 것이다. 매뉴얼에 나오는 광범위한 내용은 다루지 않고 기본적인 것들에 관심을 두기로 한다.

4.1 긴 명령어를 여러 라인으로 표시하기

Makefile을 작성할 때 명령어가 한 줄을 넘어간다고 가정하자. 이때 그냥 줄줄이 적는다면 읽기도 힘들고, 작성하는 사람도 조금 찜찜하다. 이때 '\' 문자를 이용해서 여러 라인으로 나타낼 수 있다. 이미 C언어에 익숙한 사람이라면 낯익은 기호일 것이다. 아래의 예제를 보자.

[예제 10]

```
OBJ = shape.o \  
rectangle.o \  
circle.o \  
line.o \  
bezier.o
```

위의 예제는 `OBJ = shape.o rectangle.o circle.o line.o bezier.o` 라는 문장을 여러 라인으로 표시한 것이다.

4.2 확장자 규칙의 이용

두번째 장에서 확장자 규칙에 대해서 많이 설명을 했다. Makefile을 작성할 때 C, C++, tex 등의 파일은 이미 정의되어 있는 규칙을 이용하면 간단하고, 깔끔한 Makefile을 작성할 수 있다. 두번째 장에서 직접 우리가 규칙을 간단히 구현해 보기도 했는데, 이것은 확장자 규칙의 개념을 설명하기 위함이었다.

어떤 파일들이 이미 규칙으로 정해져 있는지 한번 살펴보기로 한다. 아래에 열거된 파일들은 특별히 따로 정의하지 않은 상태에서 바로 이용할 수 있는 것들이다.(GNU Make 매뉴얼에 바탕을 두고 작성되었다.)

- C 컴파일 (XX.c → XX.o)
- C++ 컴파일 (XX.cc 또는 XX.C → XX.o)
- Pascal 컴파일 (XX.p → XX.o)
- Fortran 컴파일 (XX.f 또는 XX.F → XX.o)
- Modula-2 컴파일 (XX.def → XX.sym)
- (XX.mod → XX.o)
- assembly 컴파일 (XX.s → XX.o)
- assembly 전처리 (XX.S → XX.s)
- single object file 의 링크 (XX.o → XX)
- Yacc 컴파일(?) (XX.y → XX.o)
- Lex 컴파일(?) (XX.l → XX.o)
- lint 라이브러리 생성 (XX.c → XX.ln)
- tex 파일 처리 (XX.tex → XX.dvi)
- texinfo 파일처리 (XX.texinfo 또는 XX.texi → XX.dvi)
- RCS 파일 처리 (RCS/XX,v → XX)
- SCCS 파일처리 (SCCS/XX.n → XX)

위에 정의된 파일만이 make에서 처리할 수 있는 것은 아니다. 그 밖의 파일에 대해서는 사용자가 직접 정의해 주면 얼마든지 make를 사용할 수 있다.

그럼 이전 위와 같은 파일들을 처리하기 위한 명령어는 어떤 매크로로 정의되어 있는지 알아보자. 이미 말했듯이 아래에 열거된 매크로는 재정의 가능하다. 가령 TEX = tex 이지만 대부분 TEX = latex로 재정의 되어야 할 것이다.

- AR = ar (Archive maintaining program)
- AS = as (Assembler)
- CC = cc (= cc , C compiler)
- CXX = g++ (C++ compiler)
- CO = co (extracting file from RCS)
- CPP = \$(CC) -E (C preprocessor)
- FC = f77 (Fortran compiler)
- LEX = lex (LEX processor)
- PC = pc (Pascal compiler)
- YACC = yacc (YACC processor)
- TEX = tex (TEX processor)
- TEXI2DVI = texi2dvi (Texinfo file processor)
- WEAWE = weave (Web file processor)
- RM = rm -f (remove file)

이미 두번째 장에서 밝혔지만 위의 명령어에서 사용될 FLAG(옵션)에 정의한 매크로에 대해서도 알아보기로 한다.

- ARFLAGS = (ar archiver 의 플래그) *
- ASFLAGS = (as 어셈블러의 플래그)
- CFLAGS = (C 컴파일러의 플래그) *
- CXXFLAGS = (C++ 컴파일러의 플래그) *
- COFLAGS = (co 유틸리티의 플래그)
- CPPFLAGS = (C 전처리기의 플래그)
- FFLAGS = (Fortran 컴파일러의 플래그)
- LDFLAGS = (ld 링커의 플래그) *
- LFLAGS = (lex 의 플래그) *
- PFLAGS = (Pascal 컴파일러의 플래그)
- YFLAGS = (yacc 의 플래그) *

위에서 '*'표시한 것은 자주 쓰이게 될 플래그이다. 위에서 표시한 여러 가지 매크로들을 무조건 재정의 하라는 배려에서인지, 대부분 값이 설정되어 있지 않다. 가령 C프로그램을 짤 때 CFLAGS를 재정의 해야 할 것이다.

4.3 매크로 치환

매크로를 지정하고, 그것을 이용하는 것을 이미 알고 있다. 그런데, 필요에 의해 이미 매크로의 내용을 조그만 바꾸어야 할 때가 있다. 매크로 내용의 일부만 바꾸기 위해서는 \$(MACRO_NAME:OLD=NEW)과 같은 형식을 이용하면 된다.

```
MY_NAME = Michael Jackson
YOUR_NAME = $(NAME:Jack=Jook)
```

위의 예제에서는 Jack이란 부분이 Jook으로 바뀌게 된다. 즉 YOUR_NAME 이란 매크로의 값은 Michael Jookson 이 된다. 아래의 예제를 하나 더 보기로 한다.

```
OBJ = main.o read.o write.o
SRCS = $(OBJ:.o=.c)
```

위의 예제에서는 OBJ에서 .c가 .o로 바뀌게 된다. 즉 아래와 같다.

```
SRCS = main.c read.c write.c
```

위의 예제는 실제로 사용하면 아주 편할 때가 많다. 가령 .o 파일 100개에 .c 파일이 각각 있을 때 이들을 다 적으려면 무척이나 짜증나는 일이 될 것이다.

4.4 자동 의존관계 생성

일반적인 make의 구조는 아래와 같이 target, dependency, command가 연쇄적으로 정의되어 있는 것과 같다고 하였다.

```
target : dependency
    command
    ...
```

그런데 위에서 command가 없이 타겟과 의존 관계만 표시가 되면 이는 타겟이 어느 파일에 의존하고 있는지 표시해 주는 정보의 역할을 한다. 이런 정보는 Makefile을 작성할 때 없어서는 안되는 부분이다.

그런데 일일이 이런 정보를 만든다는 것은 쉬운 일이 아니다.

이런 단조롭고 귀찮은 일을 자동으로 해주는 좋은 유틸리티가 있다. 우선 ccmakedep가 있는지 확인해 보자. *ccmakedep는 어떤 파일의 의존 관계를 자동으로 조사해서 Makefile의 뒷부분에 자동으로 붙여 주는 유틸리티이다.* ccmakedep가 없다면 cc -M XX.c 라고 해보자. 그러면 XX.c의 의존 관계가 화면에 출력됨을 알 수 있을 것이다. (ccmakedep 도 내부적으로 cc -M 을 사용한다.)

프로그램을 설치할 때 make dep 라는 것을 친 기억이 있을 것이다. 파일들의 의존 관계를 작성해 준다는 의미이다. 그럼 우리의 Makefile에도 이런 기능을 첨가해 보기로 한다.

[예제 11]

```
.SUFFIXES : .c .o
CFLAGS = -O2 -g

OBJS = main.o read.o write.o
SRCS = $(OBJS:.o=.c)

test : $(OBJS)
    $(CC) -o test $(OBJS)

dep :
    gccmakedep $(SRCS)
```

위의 Makefile을 이해할 수 있다면 이제 Makefile에 대해서 어느 정도 도가 텃다고 해도 무방할 것이다. 위의 예제에서 파일들간의 의존 관계가 없다. 그럼 이제 make dep 을 써서 자동적으로 생성시켜 보자.

```
$ make dep
$ vi(emacs) Makefile
```

Makefile의 뒷부분에 다음과 같은 내용이 붙어 있는 것을 알게 될 것이다.

[예제 12]

```
# DO NOT DELETE
main.o: main.c /usr/include/stdio.h /usr/include/features.h \
/usr/include/sys/cdefs.h /usr/include/libio.h \
/usr/include/_G_config.h io.h
read.o: read.c io.h
write.o: write.c io.h
```

main.o에는 헤더 파일들이 추가되어 있다. 이것은 헤더 파일 안에서 include하는 파일들을 모두 표현한 것이다.

4.5 다중 타겟

하나의 Makefile에서 꼭 하나의 결과만 만들어 내라는 법은 없다. 가령 결과 파일이 3개가 필요하다고 하자. 아래의 예제를 보기로 한다.

[예제 13]

```
.SUFFIXES : .c .o
CC = gcc
CFLAGS = -O2 -g

OBJ1 = main.o test1.o <- 각각의 매크로를 정의
OBJ2 = main.o test2.o
OBJ3 = main.o test3.o
SRCS = $(OBJ1:.o=.c) $(OBJ2:.o=.c) $(OBJ3:.o=.c)
```

```
all : test1 test2 test3 <- 여기에 주의
```

```
test1 : $(OBJ$1)
        $(CC) -o test1 $(OBJ$1)
```

```
test2 : $(OBJ$2)
        $(CC) -o test2 $(OBJ$2)
```

```
test3 : $(OBJ$3)
        $(CC) -o test3 $(OBJ$3)
```

```
dep :
        gccmakedep $(SRCS)
```

위의 프로그램은 make all 을 함으로써 동작한다. 실제로 동작시켜 보면 아래와 같다

```
$ make all (또는 make)
gcc -O2 -g -c main.c -o main.o
gcc -O2 -g -c test1.c -o test1.o
gcc -o test1 main.o test1.o <- test1 의 생성
gcc -O2 -g -c test2.c -o test2.o
gcc -o test2 main.o test2.o <- test2 의 생성
gcc -O2 -g -c test3.c -o test3.o
gcc -o test3 main.o test3.o <- test3 의 생성
```

4.6 순환 make

규모가 큰 프로그램들은 파일들이 하나의 디렉토리에 있지 않는 경우가 많다. 여러 개의 서브 시스템이 전체 시스템을 구성한다고 가정하면 각 서브시스템에 Makefile이 존재한다. (서브시스템 = 서브디렉토리) 따라서 여러 개의 Makefile을 동작시킬 필요가 있도록 Makefile을 수정해 보자.


```

subsystem:
    cd subdir; $(MAKE) .....(1)

subsystem:
    $(MAKE) -C subdir .....(2)

```

위의 예제에서 (1)과 (2)는 동일한 명령을 수행한다. 우리가 만들 시스템의 타겟이 subsystem 이라면 우선 subdir로 이동하여 거기에 있는 Makefile을 동작시키게 된다. MAKE라는 것은 그냥 make라는 명령어를 표시하는 매크로일 뿐이다. 그럼 완전한 예제를 한번 구성해 보기로 한다.

[예제 14]

```

.SUFFIXES : .c .o
CC = cc
CFLAGS = -O2 -g

all : DataBase Test

DataBase:
    cd db ; $(MAKE)    <- db 로 이동해서 make 실행

Test:
    cd test ; $(Make)  <- test 로 이동해서 make 실행

```

위의 예제에서 db, test 디렉토리에 있는 Makefile은 지금까지 우리가 공부했던 Makefile과 거의 흡사하다고 가정하자. 그럼 위의 Makefile을 실행시켜 본다.

```

$ make
cd db ; make
make[1]: Entering directory `/home/user/db'
gcc -O2 -g -c DBopen.c -o DBopen.o
gcc -O2 -g -c DBread.c -o DBread.o

```

```
gcc -O2 -g -c DBwrite.c -o DBwrite.o
make[1]: Leaving directory `/home/user/db'
cd test ; make
make[1]: Entering directory `/home/user/test'
gcc -O2 -g -c test.c -o test.o
make[1]: Leaving directory `/home/user/test'
```

위의 가상 실행을 보면 우선 db로 가서 거기의 Makefile을 수행시키고, 다음에는 test로 가서 Makefile을 실행시킴을 볼 수 있다. 우선은 단순히 컴파일만 시켰는데, 다르게 한번 생각해 보자. db 디렉토리에서의 최종 타겟으로 가령 db.a을 만들어 내고 test 디렉토리에서 이를 링크 시킨다고 생각하면 꽤 괜찮은 시나리오가 될 것이다. 위에서 1이라고 나타난 것은 현재의 레벨을 의미한다. 원래 디렉토리의 레벨이 0이고, 여기서는 레벨이 하나 더 내려갔으므로 1이라고 표시된 것이다.

4.7 불필요한 재컴파일 막기

의존 관계 규칙에 의해 하나가 바뀌면 그에 영향 받는 모든 파일이 바뀐다. 그러나 다른 파일들에게 아무 영향을 주지 않도록 수정하였는데도 재컴파일을 시도한다면 시간 낭비가 될 수도 있다. 가령 모든 .c 파일에서 include 하는 헤더 파일에서 새로운 #define PI 3.14 라고 정의를 했다고 가정하자. 그리고 PI라는 것은 아무 곳에서도 사용을 하지 않는다.

이때는 'make -t' 라고 해보자. *-t 는 touch를 의미하는 옵션으로써 컴파일을 하지 않는 대신 파일의 생성 날짜만 가장 최근으로 바꾸어 놓는다.* 새로 컴파일 된 것처럼 처리를 하는 것이다.

4.8 make 주요 옵션

make에서 거의 모든 것은 Makefile내부에서 모두 지정을 할 수 있다. 그중 일부를 make의 실행 시에 옵션으로 통해서 줄 수도 있다. 수많은 옵션중에서 기억해 둘 만한 것 몇 가지만 소개하기로 한다.

-C dir

위에서도 밝혔듯이 Makefile을 계속 읽지 말고 우선은 dir로 이동하라는 것이다. 순환 make에 사용된다.

-d

Makefile을 수행하면서 디버깅 정보를 포함하여 모든 정보를 출력해 준다. (-debug) 출력량이 매우 많다. 결과를 파일로 저장해서 읽어보면 make의 동작을 대충 이해할 수 있다.

-h

옵션에 관한 도움말을 출력한다. (-help)

-f file

file에 해당하는 파일을 Makefile로써 취급한다. (-file)

-r

내장하고 있는 각종 규칙(Suffix rule 등)을 없는 것으로 (-no-builtin-rules)간주한다. 따라서 사용자가 규칙을 새롭게 정의해 주어야 한다.

-t

파일의 생성 날짜를 현재 시간으로 갱신한다. (-touch)

-v

make의 버전을 출력한다. (-version)

-p

make에서 내부적으로 세팅되어 있는 값들을 출력한다. (-print-data-base)

-k

make는 에러가 발생하면 도중에 실행을 포기하게 되는데 (-keep-going) -k 는 에러가 나더라도 멈추지 말고 계속 진행하라는 뜻

4.9 Makefile 작성 가이드라인

make를 많이 써 본 사람은 어느 정도 자신만의 Makefile을 작성하는 일정한 스타일 같은 것이 있다. 프로그램이 짜는 사람마다 다르듯이 Makefile도 각각이다. 여기서는 그냥 가장 일반적인 가이드라인을 제시하기로 한다. 다음 장에서 Makefile의 여러 예제를 살펴보면서 다시 한번 자세히 설명할 것이다.

매크로를 잘 사용하면 Makefile이 깔끔해질 뿐 아니라, 내용의 수정도 용이하다. Makefile내에서 두번 이상 나오는 것들은 매크로로 정의해 두면 편하다. 자신의 프로그램 특성에 따라서 기존의 매크로를 재정의 하는 것도 좋다.

make에서 정의되어 있는 규칙들을 최대한 이용한다. 확장자 규칙은 무조건 이용하기를 권한다. 대체로 아래와 같이 Makefile을 구성한다.

1. 매크로 정의 부분
2. 타겟을 얻기 위한 명령어 부분
3. 의존 관계 부분

[예제 15]

```
.SUFFIXES : .c .o
CFLAGS = -g

OBJS = main.o \
read.o \           매크로 정의 부분
write.o
SRCS = $(OBJS:.o=.c)

TARGET = test

$(TARGET): $(OBJS)
    $(CC) -o $@ $(OBJS)
dep :
    gccmakedep $(SRCS)           명령어 정의 부분
new :
```

```
touch $(SRCS) ; $(MAKE)
clean :
    $(RM) $(OBJS) $(TARGET) core
```

위의 예제는 최대한 매크로를 많이 이용하려고 했기 때문에 독해하기 어려울 수도 있다.

7. make 에러

다음은 make 에 의해서 생성될 수 있는 가장 일반적인 에러들의 리스트이며 그것들이 의미하는 바와 그것들을 고치는 방법에 대한 정보이다.

때때로 make 에러들은, 특별히 명령 스크립트 라인에서 - 접두사가 있을 때나 명령 라인 옵션으로써 -k 가 있을 때, 치명적인 것이 아니다. 치명적인 에러들은 *** 문자열이 그 앞에 붙는다.

에러 메시지들은 모두 프로그램(보통 'make')의 이름이 앞에 붙거나, 에러가 makefile 안에 있는 것이라면 문제를 담고 있는 파일과 라인번호가 앞에 붙는다.

아래 테이블에서 이런 공통 접두사들이 빠져있다.

``[foo] Error NN`

``[foo] signal description'`

이런 에러들은 실제로 make 에러들이 전혀 아니다. 그들은 make 가 명령 스크립트의 일부로서 호출한 프로그램이, make 가 실패로 해석하는 0이 아닌 에러 코드 ('Error NN') 를 리턴하거나 다른 이상한 스타일로(어떤 종류의 시그널과 함께) 종료하였다는 것을 의미한다. *** 가 메시지에 붙어 있지 않으면 서브프로세스가 실패했지만 makefile 의 그 규칙이 특수 문자 - 를 앞에 달고 있어서 make 가 그 에러를 무시한 것이다.

``missing separator. Stop.'`

이것은 make 의 일반적인 "Huh?" 에러 메시지이다. 이것은 make 가 makefile 의 이 라인을 파싱하면서 완벽하게 성공하지 못했다는 것을 의미한다. 이것은 기본적으로 "문법 에러 (syntax error)" 를 의미한다. 이런 메시지가 나오는 가장 일반적인 이유들 중의 하나는 여러분이 (또는 많은 MS-Windows 에디터들의 경우와 비슷하게, 여러분의 에디터가) TAB 문자 대신에 공백들로 명령 스크립트들을 들여쓰기하려고 하는 것이다. 명령 스크립트에 있는 모든 라인은 반드시 TAB 문자로 시작하는 것을 기억하자. 8개의 공백은 의미가 없다.

``commands commence before first target. Stop.'`

``missing rule before commands. Stop.'`

이것은 makefile 에서 처음으로 나오는 것이 명령 스크립트의 일부분인 것처럼 보인다는 것을 의미한다: 이것은 TAB 문자로 시작하고 합법적인 make 명령(변수 할당과 같은)처럼 보이지 않는다. 명령 스크립트들은 항상 어떤 타겟과 연결되어 있어야 한다. 두번째 형태는 그 라인이 첫번째 공백문자가 아닌 문자로써 세미콜론을 가진다면 생성된다; make 는 이것을, 어떤 규칙의 "target: dependency" 섹션을 그냥 떠났다는 것으로 해석한다.

```
`No rule to make target `xxx`.'
```

```
`No rule to make target `xxx`, needed by `yyy`.'
```

이것은 make 가 타겟을 빌드할 필요가 있다고 판단했지만 makefile 안에서 그렇게 하는 것에 대한, 명시적 또는 묵시적(디폴트 규칙 데이터베이스를 포함해서) 규칙들(instructions)도 찾을 수 없다는 것을 의미한다. 그 파일이 빌드되기를 원한다면 그 타겟이 빌드되는 방법을 설명하는 규칙을 추가할 필요가 있다. 이 문제의 다른 가능성은 makefile 을 오자(그 파일 이름이 잘못되었다)했거나 소스 트리가 잘못된 경우이다(그 파일이 빌드될 것으로 생각된 것이 아니고 단지 종속물이다).

```
`No targets specified and no makefile found. Stop.'
```

```
`No targets. Stop.'
```

전자는 명령행에서 빌드될 타겟을 하나도 제공하지 않았고 make 가 읽어들이 makefile 들을 찾을 수 없다는 것을 의미한다. 후자는 어떤 makefile 들이 찾아졌으나 디폴트 타겟이 없고 어떤 것도 명령행에서 주어지지 않았다는 것을 의미한다. 이들 경우에 GNU make 는 아무것도 하지 않는다.

```
`Makefile `xxx` was not found.'
```

```
`Included makefile `xxx` was not found.'
```

명령행에서 주어진 makefile (첫번째 형태) 또는 포함된 makefile (두번째 형태) 가 없다.

```
`warning: overriding commands for target `xxx`'
```

```
`warning: ignoring old commands for target `xxx`'
```

GNU make 는 타겟 하나에 대해서 단 한번만 명령들이 지정되는 것을 허락한다(더블-콜론

규칙들을 제외하고). 이미 명령들을 가지도록 정의된 타겟에 대해서 명령들을 다시 주면 이 경고가 발행되고 두번째 명령들은 첫번째 것을 오버라이드할 것이다.

``Circular xxx<- yyy dependency dropped.'`

`make` 가 종속성 그래프에서 루프를 발견했다는 것을 의미한다: 타겟 `xxx` 의 종속물 `yyy`, 그리고 이것의 종속물들 등등 을 추적한 후 그들중 하나가 `xxx` 에 다시 종속한다는 의미이다.

``Recursive variable `xxx' references itself (eventually). Stop.'`

이것은 확장될 때 자기자신(`xxx`)을 참조할 일반 (재귀적인) `make` 변수 `xxx` 를 정의했다는 것을 의미한다. 이것은 허용되지 않는다; 단순-확장 변수 (`:=`) 를 사용하든지 아니면 추가 연산자 (`+=`) 를 사용하자.

``Unterminated variable reference. Stop.'`

이것은, 변수나 함수 참조에서 적절하게 닫는 괄호나 종괄호를 제공하는 것을 잊었다는 것을 의미한다.

``insufficient arguments to function `xxx'. Stop.'`

이것은 이 함수에 대해서 필요한 개수의 매개변수들을 제공하지 않았다는 것을 의미한다. 매개변수들의 설명에 대해서는 그 함수의 문서를 보자.

``missing target pattern. Stop.'`

``multiple target patterns. Stop.'`

``target pattern contains no `%'. Stop.'`

이들은 잘못된 정적 패턴 규칙들에 대해서 생성된다. 첫번째는 규칙의 타겟 섹션에 어떤 패턴도 없다는 것을 의미하고, 두번째는 타겟 섹션에 다수의 패턴들이 있다는 것을 의미하며, 세번째는 타겟이 패턴 문자 (%) 를 담고 있지 않다는 것을 의미한다.